개발환경 세팅 가이드

새 Windows PC 에서 PowerShell 로 도구 설치 → 저장소 받기 → 백엔드·FO·BO 를 브라우저로 띄우기까지 · 2026-10-08 실측 · 2026-10-16 새 노트북(Windows 10)에서 처음부터 재검증

끝나면 이렇게 된다

이름	기술	주소	폴더
백엔드(API)	Spring Boot 3 · Java 21	`http://localhost:8080/api`	`backend`
FO(사용자 화면)	Next.js 16	`http://localhost:3000`	`frontend/nextjs`
BO(관리자 화면)	React · Vite	`http://localhost:5173`	`frontend/react_js`

DB 는 PC 에 설치하지 않는다. 회사 DB 서버의 **개발 DB**(`freelancer_develop`)에 붙는다. Redis 만 PC 에 설치한다.

시작 전에 받아 둘 것

항목	설명
`application.yml`	**담당자에게 파일로 받는다.** 백엔드 설정(DB·Redis·메일·JWT 등 비밀값)이 들어 있어 저장소에 없다. 절대 커밋하지 않는다
Redis 비밀번호	받은 `application.yml` 의 `spring.data.redis.password` 값. PC 의 Redis 에 같은 비밀번호를 건다(1-3)
BO 관리자 계정	BO 로그인용 아이디·비밀번호. 담당자에게 받는다
네트워크	인터넷만 되면 된다. 회사 DB 서버(`db.estsw.co.kr:3306`)는 사내망이 아니어도 접속된다. 사무실에서는 숨김 와이파이 `est_server` (0단계, 비밀번호는 담당자)
디스크	C: 여유 5GB 이상 권장(도구 설치 + `node_modules` 2개 + Gradle 캐시)

전체 흐름

단계	내용	소요(실측)
0 와이파이	사무실 와이파이 `est_server` 등록·연결(숨김 와이파이)	2~3분
1 도구 설치	실행 정책 · winget 확인(없으면 설치), winget 으로 Git · JDK 21 · Node.js 22 · Redis, Redis 비밀번호	10~20분 · winget 설치 시 +5분
2 저장소·설정	`git clone` (NextJs 브랜치), `application.yml` 넣기	1~2분
3 백엔드	`gradlew bootRun`	첫 실행 수 분 · 이후 약 25초
4 FO	`npm ci` → `npm run dev`	설치 약 2분 · 기동 약 10초
5 BO	`npm ci` → `npm run dev`	설치 1~2분 · 기동 약 16초

0 사무실 와이파이 est_server 연결

사무실 개발 PC 는 `est_server` 와이파이에 연결해 둔다. 연결하면 PC 가 `192.168.2.x` 주소를 받아 **운영 서버(`192.168.2.104`)와 같은 망**이 된다. 개발 자체(코드 받기 · DB 접속)는 다른 인터넷에서도 되지만, **운영 서버 접속 · 배포는 이 와이파이에서만** 된다(「운영 배포 가이드」 0단계와 같은 절차).

> **`est_server` 는 숨김 와이파이다.** 와이파이 목록에 나타나지 않으므로 **처음 한 번은 직접 등록**해야 한다. 비슷한 이름(`est_5G` · `est_2.4G` 등)은 **다른 망**이다. 비밀번호는 담당자에게 받는다(문서·저장소에는 적지 않는다).

0-1. 등록 방법 ① — Windows 설정 화면

1. `Win + I` → **네트워크 및 인터넷 → Wi-Fi → 알려진 네트워크 관리**
2. **네트워크 추가**(Windows 10 은 「새 네트워크 추가」)
3. 입력:

칸	값
네트워크 이름	`est_server` (밑줄 포함, 대소문자 그대로)
보안 유형	**WPA2-개인 AES** (공유기가 WPA2/WPA3 겸용이라 WPA3-개인 도 된다)
보안 키	담당자에게 받은 비밀번호
☑ 자동으로 연결	체크
☑ 네트워크가 브로드캐스팅하지 않는 경우에도 연결	**반드시 체크** — 숨김 와이파이라 이게 없으면 못 붙는다

4. **저장** → 잠시 뒤 자동 연결된다. 안 되면 작업 표시줄 와이파이 목록 맨 아래 「숨겨진 네트워크」에서 연결

이 화면 절차는 캡처 없이 글로만 적었다.

0-1. 등록 방법 ② — 터미널(PowerShell) 한 번에

Windows PowerShell — 등록·연결 명령 (PC 당 한 번)

```
PS C:\Users\ESTSOFT> $xml = '<WLANProfile xmlns="http://www.microsoft.com/networking/WLAN/profile/v1"><name>est_server</
name><SSIDConfig><SSID><name>est_server</name></SSID><nonBroadcast>true</nonBroadcast></SSIDConfig><connectionType>ESS</
connectionType><connectionMode>auto</connectionMode><MSM><security><authEncryption><authentication>WPA2PSK</authenticati
on><encryption>AES</encryption><useOneX>false</useOneX></authEncryption><sharedKey><keyType>passPhrase</keyType><protect
ed>false</protected><keyMaterial>비밀번호</keyMaterial></sharedKey></security></MSM></WLANProfile>'
PS C:\Users\ESTSOFT> $xml | Out-File -Encoding utf8 "$env:TEMP\est_server.xml"
PS C:\Users\ESTSOFT> netsh wlan add profile filename="$env:TEMP\est_server.xml"
PS C:\Users\ESTSOFT> Remove-Item "$env:TEMP\est_server.xml"
PS C:\Users\ESTSOFT> netsh wlan connect name=est_server
```

첫 줄의 **비밀번호** 를 실제 비밀번호로 바꾼 뒤 실행한다(나머지는 그대로). 넷째 줄은 비밀번호가 든 임시 파일을 지운다.

Windows PowerShell — 실측(시험용 이름)

```
PS C:\Users\ESTSOFT> $xml = '<WLANProfile xmlns="http://www.microsoft.com/networking/WLAN/profile/v1"><name>est_server_t
est</name><SSIDConfig><SSID><name>est_server</name></SSID><nonBroadcast>true</nonBroadcast></SSIDConfig><connectionType>
ESS</connectionType><connectionMode>manual</connectionMode><MSM><security><authEncryption><authentication>WPA2PSK</authe
ntication><encryption>AES</encryption><useOneX>false</useOneX></authEncryption><sharedKey><keyType>passPhrase</keyType><
protected>false</protected><keyMaterial>비밀번호</keyMaterial></sharedKey></security></MSM></WLANProfile>'
PS C:\Users\ESTSOFT> $xml | Out-File -Encoding utf8 "$env:TEMP\est_server.xml"
PS C:\Users\ESTSOFT> netsh wlan add profile filename="$env:TEMP\est_server.xml"
Profile est_server_test is added on interface Wi-Fi.
PS C:\Users\ESTSOFT> Remove-Item "$env:TEMP\est_server.xml"
PS C:\Users\ESTSOFT> netsh wlan show profile name=est_server_test | Select-String "SSID name|broadcast|Authentication"

        Network broadcast  : Connect even if this network is not broadcasting
    SSID name              : "est_server"
    Authentication         : WPA2-Personal
    Authentication         : WPA2-Personal
PS C:\Users\ESTSOFT> netsh wlan delete profile name=est_server_test
Profile "est_server_test" is deleted from interface "Wi-Fi".
```

실측은 지금 연결을 끊지 않으려고 이름만 `est_server_test` 로 수동 연결로 바꿔 등록 → 확인 → 삭제했다. 비밀번호는 가렸다.

0-2. 확인

Windows PowerShell

```
PS C:\Users\ESTSOFT> netsh wlan show interfaces | Select-String " SSID"
    SSID                   : est_server
PS C:\Users\ESTSOFT> (Get-NetIPAddress -InterfaceAlias Wi-Fi -AddressFamily IPv4).IPAddress
192.168.2.105
```

SSID 가 est_server 이고 IP 가 192.168.2.x 면 연결된 것이다. 다음부터는 와이파이 목록에서 고르거나 `netsh wlan connect name=est_server` .

1 도구 설치

1-0. PowerShell 준비 — 실행 정책 · winget

시작 메뉴 → 「PowerShell」. 새 PC 는 **스크립트 실행이 막혀 있어**(실행 정책 Restricted) 이대로면 Node 설치 뒤 `npm` 이 「스크립트를 실행할 수 없으므로…」로 실패한다. 먼저 한 번 푼다(관리자 권한 불필요).

Windows PowerShell — 실행 정책 (PC 당 한 번)

```
PS C:\Users\ESTSOFT> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned -Force
PS C:\Users\ESTSOFT> Get-ExecutionPolicy
RemoteSigned
```

RemoteSigned 가 나오면 된다. 안 하고 넘어가면 1-2 의 `npm -v` 에서 `npm.ps1 cannot be loaded because running scripts is disabled on this system` 이 난다(2026-10-16 새 노트북에서 실제로 났다).

다음으로 `winget --version` 을 친다. 버전(`v1.x`)이 나오면 1-1 로 넘어간다. **Windows 11 · 업데이트된 Windows 10 에는 들어 있지만, 오래 업데이트 안 된 Windows 10 에는 없다**(「'winget' 용어가 … 인식되지 않습니다」). 그때만 아래를 한다.

winget 이 없을 때 — ① 관리자: Windows PowerShell

```
PS C:\WINDOWS\system32> Install-PackageProvider -Name NuGet -Force
PS C:\WINDOWS\system32> Install-Module -Name Microsoft.WinGet.Client -Force -Repository PSGallery
```

관리자 권한 창(시작 메뉴 → PowerShell 마우스 오른쪽 → 관리자 권한으로 실행)에서 두 줄. winget 설치 도구(PowerShell 모듈)를 받는다.

winget 이 없을 때 — ② 일반 Windows PowerShell

```
PS C:\Users\ESTSOFT> Repair-WinGetPackageManager -Latest
PS C:\Users\ESTSOFT> winget --version
v1.29.380
```

일반 창에서 실행한다(약 300MB 를 받아 2~5분). 관리자 창에서 `-AllUsers` 를 붙이면 「액세스가 거부되었습니다」로 실패했다 — 붙이지 않는다. 2026-10-16 새 노트북(Windows 10 22H2, 앱 설치 관리자 1.0.30251 → 1.29.380) 실측.

1-1. 설치 명령

winget 이 되면 아래 네 줄을 차례로 실행한다.

Windows PowerShell — 설치 명령

```
PS C:\Users\ESTSOFT> winget install --id Git.Git -e
PS C:\Users\ESTSOFT> winget install --id Oracle.JDK.21 -e
PS C:\Users\ESTSOFT> winget install --id OpenJS.NodeJS.22 -e
PS C:\Users\ESTSOFT> winget install --id Redis.Redis -e
```

이 네 줄이 설치 명령이다. 처음 한 번 원본 약관 동의(Y), 패키지마다 사용권 동의를 물으면 Y. Git · JDK · Redis 는 **「이 앱이 디바이스를 변경하도록 허용」 창에서 「예」**. 끝나면 **PowerShell 창을 닫고 새로 연다**(PATH 반영).

패키지	용도
`Git.Git`	저장소 받기 · 커밋
`Oracle.JDK.21`	백엔드 빌드·실행. **반드시 21**(프로젝트가 Java 21 로 고정돼 있다)
`OpenJS.NodeJS.22`	FO · BO 실행. npm 도 같이 설치된다. 지금은 압축본으로 설치돼 `%LOCALAPPDATA%\Microsoft\WinGet\Packages\…` 아래에 들어간다(정상)
`Redis.Redis`	로그인 토큰 저장소. Windows 서비스로 설치돼 PC 를 켜면 자동으로 돈다

1-1b. (선택) Python · Chrome — `docs/도구` 스크립트를 쓸 때

DB 확인·백업·배포 보조 스크립트(`docs/도구/*.py`)를 돌릴 사람만 설치한다. 개발 서버 띄우기에는 필요 없다.

Windows PowerShell

```
PS C:\Users\ESTSOFT> winget install --id Python.Python.3.12 -e
PS C:\Users\ESTSOFT> winget install --id Microsoft.VCRedist.2015+.x64 -e
# 새 창에서
PS C:\Users\ESTSOFT> python --version
Python 3.12.10
```

설치 전에는 `python` 이 Microsoft Store 바로가기라 실행하면 Store 가 열린다. 둘째 줄(Visual C++ 런타임)이 없으면 `pymupdf` 가 `ImportError: DLL load failed` 로 실패한다(새 PC 에는 없다 — 2026-10-16 실측).

Chrome(Playwright 스크립트가 `channel='chrome'` 으로 씀)은 보통 이미 깔려 있다. 없으면 브라우저로 `google.com/chrome` 에서 받는다 — `winget install Google.Chrome` 은 「설치 관리자 해시가 일치하지 않습니다」로 자주 실패한다(2026-10-16 실측).

1-2. 설치 확인

Windows PowerShell

```
PS C:\Users\ESTSOFT> winget list --id Git.Git -e
이름 장치 ID 버전      원본
-----------------------------
Git  Git.Git 2.55.0.5 winget

PS C:\Users\ESTSOFT> winget list --id Oracle.JDK.21 -e
이름                                        장치 ID       버전       원본
-------------------------------------------------------------------------------
Java(TM) SE Development Kit 21.0.12 (64-bit) Oracle.JDK.21 21.0.12.0 winget

PS C:\Users\ESTSOFT> winget list --id OpenJS.NodeJS.22 -e
이름       장치 ID          버전     원본
-----------------------------------------
Node.js 22 OpenJS.NodeJS.22 22.23.2 winget

PS C:\Users\ESTSOFT> winget list --id Redis.Redis -e
이름            장치 ID     버전     원본
-----------------------------------------
Redis on Windows Redis.Redis 3.0.504 winget
```

2026-10-16 새 노트북에서 설치 직후 찍었다. 「버전」 칸에 값이 있으면 설치된 것이다(나중에 「사용 가능」 칸이 생기면 더 새 버전이 있다는 안내일 뿐, 올리지 않아도 된다).

1-2. (이어서) 버전 확인 — 새 창에서

Windows PowerShell

```
PS C:\Users\ESTSOFT> git --version
git version 2.55.0.windows.5
PS C:\Users\ESTSOFT> java -version
java version "21.0.12" 2026-07-21 LTS
Java(TM) SE Runtime Environment (build 21.0.12+7-LTS-205)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.12+7-LTS-205, mixed mode, sharing)
PS C:\Users\ESTSOFT> node -v
v22.23.2
PS C:\Users\ESTSOFT> npm -v
10.9.8
```

Java 는 **21.x**, Node.js 는 **v22.x** 면 된다. 「명령을 찾을 수 없음」이면 창을 새로 열었는지 확인한다. `npm -v` 만 「스크립트를 실행할 수 없으므로」면 1-0 의 실행 정책을 안 한 것이다.

1-3. Redis 비밀번호 맞추기

받은 `application.yml` 에 Redis 비밀번호가 있다. PC 의 Redis 에 같은 비밀번호를 건다(한 번만).

관리자: Windows PowerShell

```
PS C:\WINDOWS\system32> Add-Content "C:\Program Files\Redis\redis.windows-service.conf" "requirepass 비밀번호"
PS C:\WINDOWS\system32> Restart-Service Redis
```

관리자 권한 PowerShell 에서 실행한다(시작 메뉴 → PowerShell 마우스 오른쪽 → 관리자 권한으로 실행). 성공하면 아무것도 출력하지 않는다.「비밀번호」자리에 `application.yml` 의 `spring.data.redis.password` 값을 넣는다.

확인 — 일반 PowerShell 에서:

Windows PowerShell

```
PS C:\Users\ESTSOFT> Get-Service Redis

Status   Name               DisplayName
------   ----               -----------
Running  Redis              Redis

PS C:\Users\ESTSOFT> & "C:\Program Files\Redis\redis-cli.exe" ping
NOAUTH Authentication required.
PS C:\Users\ESTSOFT> & "C:\Program Files\Redis\redis-cli.exe" -a 비밀번호 ping
PONG
```

비밀번호를 걸어 둔 Redis 는 그냥 ping 하면 NOAUTH, 비밀번호를 주면 PONG 이다. 실제 비밀번호 대신「비밀번호」로 적었다. 2026-10-16 새 노트북에서 위 두 줄을 실행한 뒤 확인한 결과다.

보이는 줄	의미
`Running Redis`	Redis 서비스가 돌고 있다
`NOAUTH Authentication required.`	비밀번호가 걸려 있다(정상)
`PONG`	비밀번호가 맞다. 백엔드도 이 비밀번호로 붙는다

2 저장소 받기 · 설정 파일 넣기

> 2026-10-16 새 노트북에서 1단계부터 이 순서대로 다시 실행해 확인했다. 새 PC 에는 `C:\dev` 폴더가 없으므로 **첫 줄로 만든다**(없이 `cd C:\dev` 하면 「경로가 존재하지 않으므로 찾을 수 없습니다」).

Windows PowerShell

```
PS C:\Users\ESTSOFT> mkdir C:\dev
PS C:\Users\ESTSOFT> cd C:\dev
PS C:\dev> git clone -b NextJs https://github.com/EstSoftPart3/Freelancer_Service.git
Cloning into 'Freelancer_Service'...

Updating files: 100% (8053/8053), done.

PS C:\dev> cd Freelancer_Service
PS C:\dev\Freelancer_Service> Copy-Item "$HOME\Downloads\application.yml" backend\src\main\resources\application.yml
PS C:\dev\Freelancer_Service> git status --short
# (아무것도 안 나오면 정상 — application.yml 은 git 이 무시하는 파일이다)
```

`$HOME\Downloads\application.yml` 은 담당자에게 받은 파일을 내려받은 위치의 예시다. 실제 위치로 바꾼다.

명령 · 줄	의미
`-b NextJs`	개발 기준 브랜치. **빼먹으면 main 이 받아진다**(예전 버전)
`Updating files: 100% … done.`	받기 완료(약 8,000개 파일)
`Copy-Item … application.yml`	받은 설정 파일을 `backend\src\main\resources\` 에 넣는다. 파일 이름은 정확히 `application.yml`
`git status --short` 출력 없음	설정 파일이 git 에 잡히지 않는다(올바른 위치·이름). 무언가 나오면 위치·이름을 다시 확인

> FO 의 `frontend\nextjs\.env.local` 은 **만들지 않아도 된다**(비어 있으면 기본값으로 동작). 학력 검색(커리어넷) 키가 필요할 때만 `.env.example` 을 복사해 채운다.

3 백엔드 띄우기

Windows PowerShell

```
PS C:\dev\Freelancer_Service> cd backend
PS C:\dev\Freelancer_Service\backend> .\gradlew.bat bootRun
> Task :compileJava

> Task :processResources
> Task :classes
> Task :resolveMainClassName

> Task :bootRun

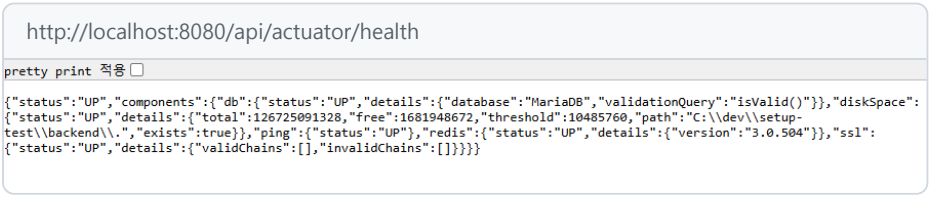
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.4.4)
  ... (중략) ...
2026-10-08T15:14:15 [main] INFO
               com.example.demo.DemoApplication - The following 1 profile is active: "dev"
  ... (중략) ...
2026-10-08T15:14:36 [main] INFO
               o.s.b.w.e.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/api'
2026-10-08T15:14:36 [main] INFO
               com.example.demo.DemoApplication - Started DemoApplication in 23.341 seconds (process running for 24.10
7)
```

마지막 줄 **Started DemoApplication** 이 나오면 기동 완료다. 이 창은 **닫지 않고** 둔다(닫으면 서버가 꺼진다). 처음 실행은 Gradle 이 라이브러리를 내려받느라 몇 분 더 걸린다.

보이는 줄	의미
`profile is active: "dev"`	개발 설정으로 떴다(운영이면 "prod")
`Tomcat started on port 8080 … '/api'`	주소는 `http://localhost:8080/api`
`Started DemoApplication in … seconds`	기동 완료

브라우저 확인 — `http://localhost:8080/api/actuator/health`



맨 앞이 `"status":"UP"` 이고 `db` · `redis` 도 `UP` 이면 DB 와 Redis 연결까지 정상이다.

4 FO(사용자 화면) 띄우기

Windows PowerShell

```
PS C:\Users\ESTSOFT> cd C:\dev\Freelancer_Service\frontend\nextjs
PS C:\dev\Freelancer_Service\frontend\nextjs> npm ci
npm warn deprecated lodash.isequal@4.5.0: This package is deprecated. Use require('node:util').isDeepStrictEqual
instead.
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead
added 699 packages, and audited 700 packages in 2m
245 packages are looking for funding
  run `npm fund` for details
33 vulnerabilities (4 low, 5 moderate, 22 high, 2 critical)
To address issues that do not require attention, run:
  npm audit fix
To address all issues (including breaking changes), run:
  npm audit fix --force
Run `npm audit` for details.

PS C:\dev\Freelancer_Service\frontend\nextjs> npm run dev

> nextjs@0.1.0 dev
> next dev

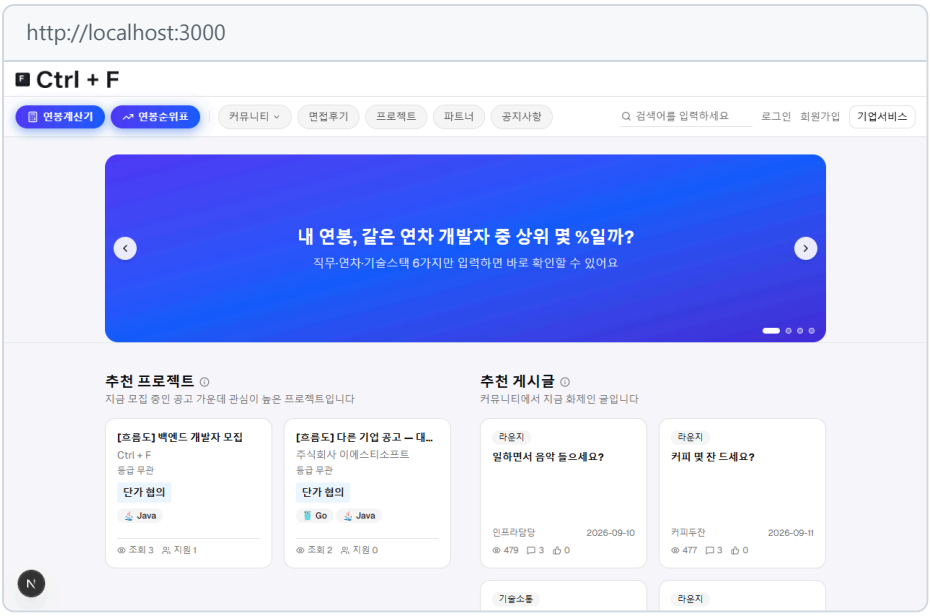
▲ Next.js 16.2.6 (Turbopack)
- Local:         http://localhost:3000
- Network:       http://172.19.0.1:3000
✓ Ready in 10.6s

○ Compiling / ...
 GET / 200 in 26.0s (next.js: 21.8s, proxy.ts: 907ms, application-code: 3.3s)
 GET / 200 in 947ms (next.js: 50ms, proxy.ts: 71ms, application-code: 827ms)
```

새 PowerShell 창에서 한다(백엔드 창은 그대로). npm warn · vulnerabilities 줄은 경고일 뿐 진행에 지장 없다.

> **`npm install` 은 따로 하지 않는다.** `npm ci` 가 `package-lock.json` 에 적힌 버전 그대로 설치하는 명령이라 이것 하나면 된다. `npm install` 은 lock 파일을 고쳐 git 에 변경이 생길 수 있다(새 패키지를 추가할 때만 `npm install 패키지명`). 저장소 맨 위와 `frontend\` 바로 아래의 `package.json` 은 예전 Vue 시절 잔재라 설치하지 않는다.

브라우저 확인 — `http://localhost:3000`



추천 프로젝트·게시글이 보이면 FO 가 백엔드(개발 DB)에서 데이터를 받아 온 것이다. 왼쪽 아래「N」동그라미는 개발 모드 표시다.

5 BO(관리자 화면) 띄우기

Windows PowerShell

```
PS C:\Users\ESTSOFT> cd C:\dev\Freelancer_Service\frontend\react_js
PS C:\dev\Freelancer_Service\frontend\react_js> npm ci
npm warn deprecated lodash.isequal@4.5.0: This package is deprecated. Use require('node:util').isDeepStrictEqual
instead.
added 451 packages, and audited 452 packages in 1m
97 packages are looking for funding
  run `npm fund` for details
33 vulnerabilities (4 low, 4 moderate, 23 high, 2 critical)
To address issues that do not require attention, run:
  npm audit fix
To address all issues (including breaking changes), run:
  npm audit fix --force
Run `npm audit` for details.

PS C:\dev\Freelancer_Service\frontend\react_js> npm run dev

> shadcn-admin@2.2.1 dev
> vite

  VITE v7.3.1  ready in 16406 ms

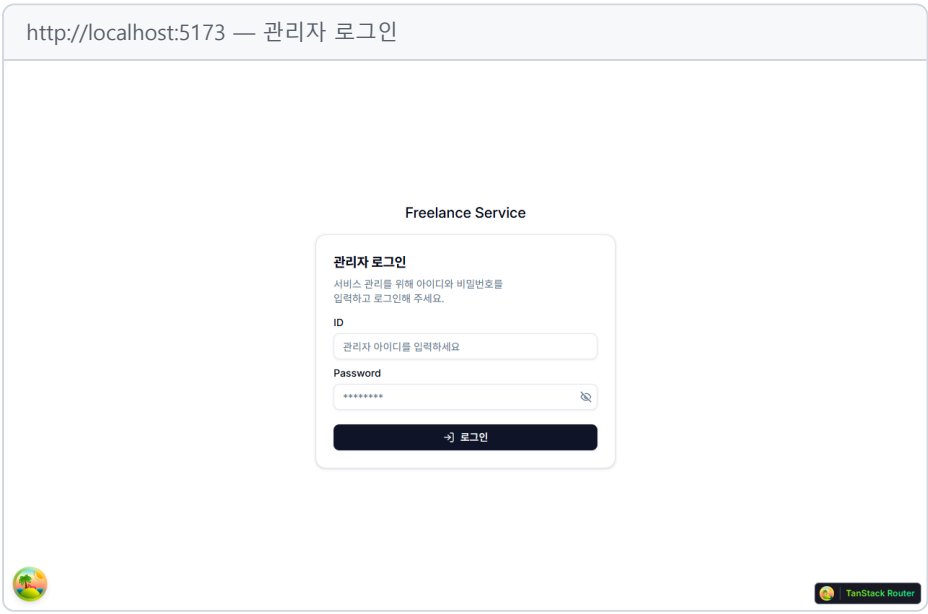
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

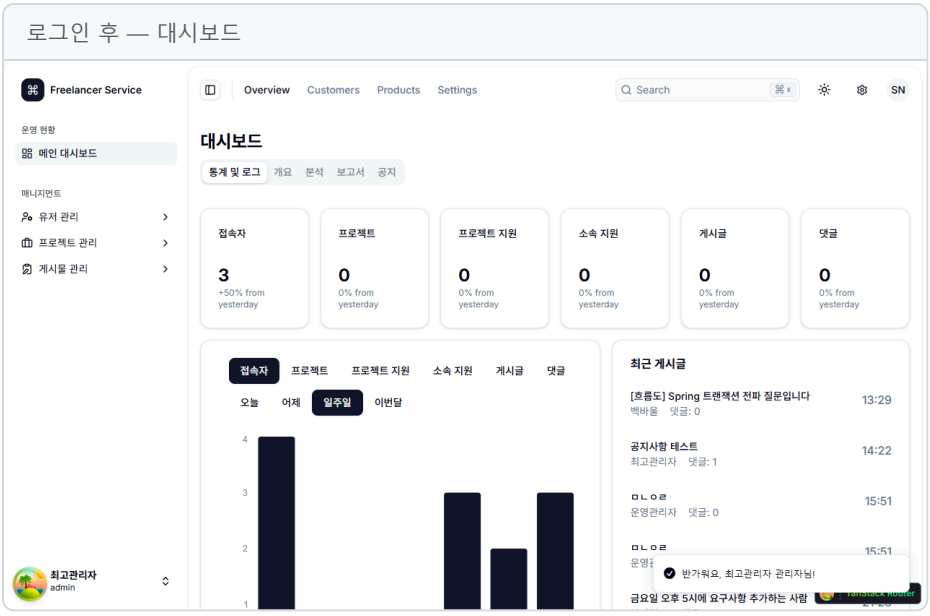
또 하나의 새 창에서 한다. 이제 창 3개(백엔드 · FO · BO)가 떠 있는 상태다.

보이는 줄	의미
`➜ Local: http://localhost:5173/`	BO 주소. **포트는 5173 이어야 한다** — 백엔드가 이 주소만 허용(CORS)한다

브라우저 확인 — `http://localhost:5173`

http://localhost:5173 — 관리자 로그인





담당자에게 받은 관리자 계정으로 로그인해 대시보드 숫자·최근 게시글이 보이면 BO ↔ 백엔드 연결까지 끝이다.

다음부터 — 매일 쓰는 법

매일 쓰는 명령

```
# 창 1 — 백엔드
PS C:\dev\Freelancer_Service\backend> .\gradlew.bat bootRun

# 창 2 — FO
PS C:\dev\Freelancer_Service\frontend\nextjs> npm run dev:clean

# 창 3 — BO
PS C:\dev\Freelancer_Service\frontend\react_js> npm run dev

# 코드 최신화 (서버를 끈 상태에서)
PS C:\dev\Freelancer_Service> git pull
```

끄는 법은 아래「서버 끄기」. `git pull` 뒤 `package-lock.json` 이 바뀌었으면 해당 폴더에서 `npm ci` 를 다시 한다.

서버 끄기

보통 — 각 창에서 Ctrl+C

창	누르면
백엔드	`Ctrl+C` 뒤 `일괄 작업을 끝내시겠습니까 (Y/N)?` 가 나오면 `Y` + Enter (`gradlew.bat` 이 배치 파일이라 묻는다)
FO · BO	`Ctrl+C` 한 번. 같은 질문이 나오면 `Y`

창을 닫아 버렸거나 포트가 계속 잡혀 있을 때 — 포트로 찾아 끄기

Windows PowerShell

```
PS C:\dev\Freelancer_Service> Get-NetTCPConnection -State Listen -LocalPort 8080,3000,5173 | Select-Object LocalPort, Owningprocess

LocalPort OwningProcess
--------- -------------
     8080         27916
     5173         24040
     3000         17512
PS C:\dev\Freelancer_Service> Get-NetTCPConnection -State Listen -LocalPort 5173 | ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }

PS C:\dev\Freelancer_Service> Get-NetTCPConnection -State Listen -LocalPort 5173 -ErrorAction SilentlyContinue
```

이 화면은 BO(5173)를 띄워 놓고 실제로 꺼 본 것이다.

명령	의미
첫 줄	8080(백엔드) · 3000(FO) · 5173(BO) 중 떠 있는 것과 그 프로세스 번호
둘째 줄	그 포트를 잡은 프로세스를 강제 종료. 성공하면 **아무것도 출력하지 않는다**. 백엔드는 `5173` 을 `8080` , FO 는 `3000` 으로 바꿔 쓴다
셋째 줄 출력 없음	포트가 비었다(꺼졌다)

백엔드를 끈 뒤에도 Gradle 이 메모리를 잡고 있으면 `backend` 폴더에서 `.\gradlew.bat --stop` 으로 정리할 수 있다 (선택).

문제 해결

증상	원인	조치
`winget` 을 찾을 수 없음	앱 설치 관리자가 옛 버전(업데이트 안 된 Windows 10)	1-0 의「winget 이 없을 때」①② 후 새 창
`npm.ps1 cannot be loaded` ·「스크립트를 실행할 수 없으므로」	실행 정책 Restricted	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned -Force` 후 새 창(1-0)
`cd C:\dev` 가「경로가 존재하지 않으므로」	새 PC 라 폴더 없음	`mkdir C:\dev` 먼저(2단계 첫 줄)
Chrome 설치가「설치 관리자 해시가 일치하지 않습니다」	winget 목록이 Google 새 설치 파일보다 늦음	브라우저로 `google.com/chrome` 에서 받는다
Python 스크립트가 `DLL load failed` (pymupdf 등)	Visual C++ 런타임 없음	`winget install --id Microsoft.VCRedist.2015+.x64 -e` (1-1b)
`java` · `node` 를 찾을 수 없음	설치 직후 같은 창	PowerShell 창을 닫고 새로 연다
`java -version` 이 1.8 · 17 등	다른 JDK 가 먼저 잡힘	JDK 21 을 설치하고 PATH 에서 21 이 앞에 오게 한다. 확인: `where.exe java`
bootRun 이 `Could not resolve placeholder`	`application.yml` 이 없거나 위치가 틀림	2단계의 경로·파일 이름 확인
bootRun 이 `NOAUTH` · `RedisConnectionFailure`	Redis 꺼짐 또는 비밀번호 불일치	1-3 확인. `Get-Service Redis` 가 Running 인지, PONG 이 나오는지
bootRun 이 DB `Communications link failure`	DB 서버에 못 닿음	인터넷 연결 확인. 공용 와이파이 등 3306 포트를 막는 네트워크면 다른 네트워크에서 다시
`Port 8080 was already in use`	백엔드가 이미 떠 있음	기존 창을 Ctrl+C 로 끄거나, `Get-NetTCPConnection -LocalPort 8080` 으로 점유 프로세스 확인
FO 일부 페이지만 404 · 이상한 화면	오래된 `.next` 캐시	FO 창에서 Ctrl+C → `npm run dev:clean`
FO 는 뜨는데 목록이 비어 있음	백엔드가 꺼져 있음	백엔드 창 확인, health 가 UP 인지
BO 로그인 시 네트워크 오류 · CORS	BO 가 5173 이 아닌 포트로 뜸	5173 을 쓰는 다른 프로그램을 끄고 BO 를 다시 띄운다
`npm ci` 실패 `EBUSY` · `EPERM`	개발 서버·편집기가 파일을 잡고 있음	해당 dev 서버를 끄고 다시 `npm ci`

배포는「운영 배포 가이드」(`docs/운영-배포-가이드.pdf`), 교육용 DB 사본은「교육용 DB 복사 가이드」를 본다.